

Assignment 1: Raster Images

Images as numpy arrays — grayscale, Bayer mosaics, HSV filters and alpha compositing.

Contents

01 Setup

02 Images in numpy

03 The twelve tasks

04 Grading and debugging

Fri Sep 25

Assignment 1 is due at 11:59 pm and is worth 5% of the course.

Assignment 2 (Ray Tracing) is also released and is due Friday
October 9.

Grace tokens

How late submissions work

- You have 8 grace tokens for the whole term.
- Each token extends a deadline by 12 hours with no penalty.
- MarkUs applies them automatically from midnight on the due date — you cannot choose which assignment uses them.
- Tokens never replenish, and weekends count.
- If you run out of tokens, a late assignment gets 0.

Example

A1 is due Friday 11:59 pm and you submit on Sunday at 4:00 pm:

```
Fri 11:59 pm → Sat 11:59 am    1 token
Sat 11:59 am → Sat 11:59 pm    1 token
Sat 11:59 pm → Sun 11:59 am    1 token
Sun 11:59 am → Sun 11:59 pm    1 token
4 tokens used · 4 left for the term
```

Submitting on MarkUs

Where

- `markus.teach.cs.toronto.edu`
- Under *Assignment 1: Raster Images*

What

- Every `.py` file in your `src/` directory.
- Nothing from `tests/` or `cgcommon/`.

When

- Friday September 25, 11:59 pm.
- After that, grace tokens are used automatically, 12 hours each.

```
$ python check_my_work.py          # aim for Validated 12/12 before you submit
```


Where to get help

- Questions about the assignment go on the A1 repository's GitHub Issues page. Answer each other's questions there too — but never post code.
- For everything else, email the TAs: csc317tas@cs.toronto.edu
- "My code doesn't work, can you find the bug?" is for tutorials and office hours — it will not be answered on Issues or by email.

01

— Setup

Getting set up

```
# 1. Clone the repository. Do not fork – forks are public.
$ git clone git@github.com:panuelosj/computer-graphics-csc317-A1-Raster_Images.git
$ cd computer-graphics-csc317-A1-Raster_Images
# 2. Once: create ./venv and install numpy, pillow, scipy, polyscope
$ ./init.sh # Windows PowerShell: .\init.ps1
# 3. In every new terminal: activate the environment
$ source venv/bin/activate # Windows: .\venv\Scripts\Activate.ps1
```

- Windows: if scripts are blocked, run `powershell -ExecutionPolicy Bypass -File init.ps1`
- Each assignment has its own venv; activate the one for the assignment you are working on.

Repository layout

`src/` is the only directory you edit

- There is one file per function you must implement.
- Each file is a stub that raises `NotImplementedError`.
- The docstring in each stub is the specification: inputs, outputs and the algorithm.

Do not edit anything else

- `cgcommon/` holds shared helpers such as image loading.
- `tests/` holds the checks and their reference data.

<code>README.md</code>	tasks and grading
<code>init.sh / init.ps1</code>	one-time venv setup
<code>main.py</code>	runs your code end-to-end
<code>run_tests.py</code>	PASS/FAIL table
<code>check_my_work.py</code>	checks weighted by marks
<code>export_expected.py</code>	test data as text + png
<code>cgcommon/</code>	shared code (do not edit)
<code>src/</code>	YOUR functions
<code>tests/</code>	unit checks (do not edit)
<code>tests/expected/</code>	readable test data
<code>data/</code>	sample input images

Running and checking your work

```
$ python main.py                # runs your functions on data/*.png → out/  
$ python main.py data/dog.png   # one specific image  
$ python run_tests.py           # PASS/FAIL per function + "Validated X/12"  
$ python run_tests.py --only demosaic  
$ python check_my_work.py       # the same checks, reported as marks
```

- `main.py` writes every result to `out/` as a `.ppm` and a `.png` preview.
- When an image check fails, `out/test_diffs/` gets three PNGs: what you produced, what was expected, and where they differ.
- Every public test input and expected output is readable in `tests/expected/`.

02

— Images in numpy

An image is a numpy array

Shape: rows, columns, channels

- RGB image: (H, W, 3)
- RGBA image: (H, W, 4)
- Grayscale or Bayer mosaic: (H, W)

Two number ranges

- Files and outputs use `uint8`, values 0–255.
- Do math in float, values 0.0–1.0 — especially for HSV.

Reading

- *Fundamentals of Computer Graphics*, 4th edition, Chapter 3.

```
# The green value at the 36th row, 89th column (Python counts from 0):  
data[35, 88, 1]  
# Every blue value on even rows and odd columns, with no loops:  
rgb[0::2, 1::2, 2]
```

Think in slices, not loops

- Operate on whole arrays with slicing, `np.where` and broadcasting instead of looping over pixels in Python.
- `uint8` arithmetic overflows silently ($200 + 100 = 44$). Convert to float first, compute, then clip to 0–255 and convert back.
- Keep track of which range you are in: 0–255 or 0–1, and hue in degrees from 0 to 360.
- Test on a tiny 3×3 array you can print — it is the fastest way to find an indexing mistake.

Gamma correction

Displays are not linear

- A pixel value of 0.5 does not produce half the light of 1.0.
- Displayed intensity follows a power law: $I = V^\gamma$, where V is the stored value in 0–1.
- Gamma correction pre-compensates by storing $V = I^{(1/\gamma)}$, so the display ends up linear.
- This is not an A1 task, but it is why averaging pixel values is not the same as averaging brightness.

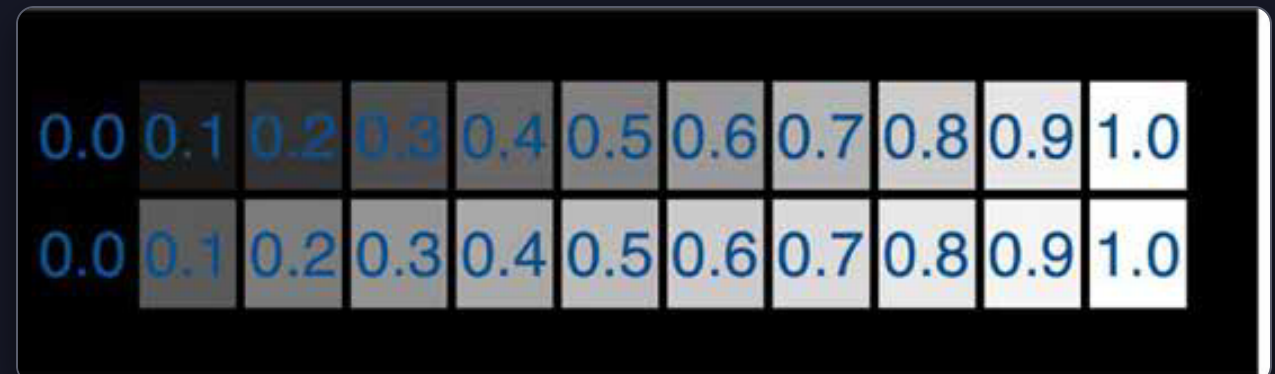
Measuring a display's gamma

Find the stored value a that looks half as bright as white:

$$\gamma = \ln 0.5 / \ln a$$

pre gamma
correction
 V_S

post gamma
correction I



The same image displayed at five gammas



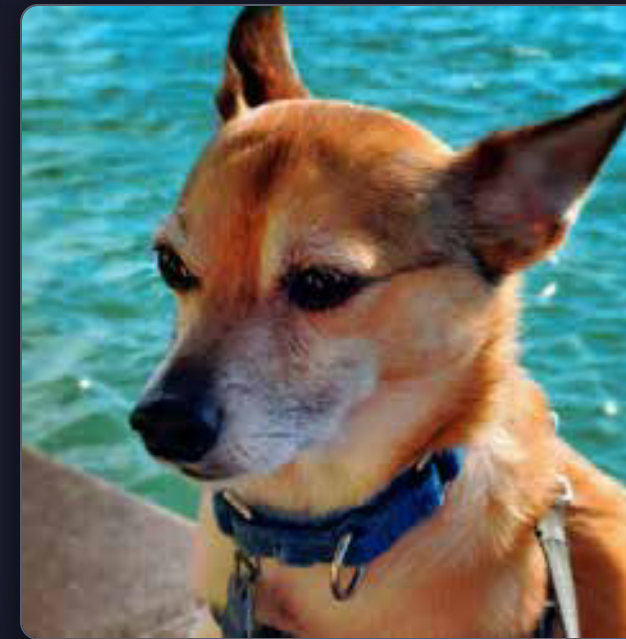
$\gamma = 0.25$



$\gamma = 0.5$



$\gamma = 1.0$ — the
linear ramp



$\gamma = 1.5$



$\gamma = 2.0$

With $I = V^\gamma$: $\gamma = 1.0$ shows the stored values as they are. A gamma below 1 brightens the midtones; a gamma above 1 darkens them.

03

— The twelve tasks

Twelve functions, 100 marks

FILE	MARKS	WHAT IT DOES	FILE	MARKS	WHAT IT DOES
rgba_to_rgb.py	10	Drop alpha channel	hsv_to_rgb.py	10	HSV → RGB
write_ppm.py	10	Write a text .ppm	hue_shift.py	10	Shift every hue
reflect.py	10	Mirror horizontally	desaturate.py	5	Reduce saturation
rotate.py	10	Rotate 90° CCW	simulate_bayer_mosaic.py	5	RGB → GBRG mosaic
rgb_to_gray.py	10	Weighted gray	demosaic.py	5	Mosaic → RGB
rgb_to_hsv.py	10	RGB → HSV	over.py	5	Porter-Duff "over"

rgba_to_rgb and write_ppm

CONCEPT

- An RGBA image has a 4th channel, alpha: how opaque each pixel is (0 transparent, 255 solid).

YOUR TASK — 10 MARKS

- `rgba_to_rgb(rgba)`: given an $(H, W, 4)$ array, return the $(H, W, 3)$ array of just the red, green and blue channels.

```
def rgba_to_rgb(rgba):  
    # rgba: (H, W, 4) → (H, W, 3)  
    # TODO: keep only channels 0, 1, 2
```

CONCEPT

- `.ppm` is a plain-text image format: a short header, then every pixel value as an integer 0–255 — open it in any editor to check it.

YOUR TASK — 10 MARKS

- `write_ppm(filename, data)`: write data to a file — P2 for a 2-D grayscale array, P3 for an $(H, W, 3)$ RGB array. Header first, then one row of values per line.

```
P3                                # "P2" if grayscale  
<width> <height>  
255  
r g b  r g b  ...  # one row per line
```

reflect and rotate

CONCEPT

- An image array is indexed [row, column]. Mirroring left ↔ right means reversing the column axis (axis 1).
- Rotating 90° swaps the two axes, so an (H, W) image becomes (W, H).

YOUR TASK — 10 + 10 MARKS

- `reflect(input)`: return the image mirrored horizontally, same shape.
- `rotate(input)`: return the image rotated 90° counter-clockwise, shape (W, H) or (W, H, C).
- Both must accept grayscale (H, W) and multi-channel (H, W, C) input.

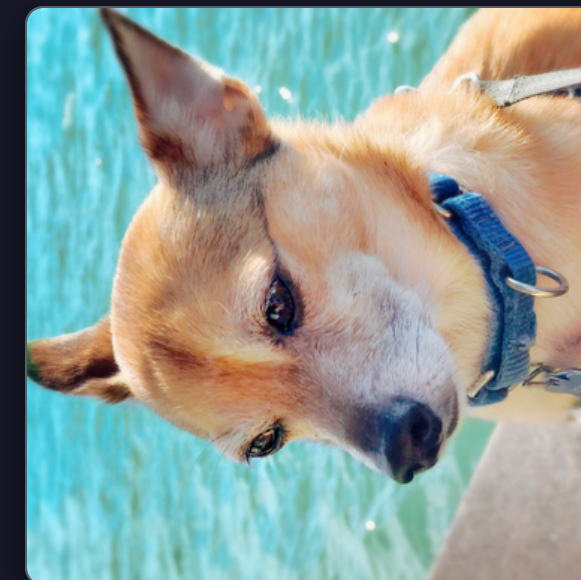
```
def rotate(input):  
    # required mapping: out[j, i] == input[i, W-1-j]  
    # slicing + an axis swap, or np.rot90
```



input



reflect



rotate

rgb_to_gray

CONCEPT

- A grayscale image has one channel: brightness only.
- The human eye is far more sensitive to green than to red or blue, so a perceptual grayscale weights green most instead of averaging the channels equally.

YOUR TASK — 10 MARKS

- `rgb_to_gray(rgb)`: turn an $(H, W, 3)$ image into an (H, W) image using the weights below.
- Compute in float, then convert back to uint8.

```
def rgb_to_gray(rgb):  
    # rgb: (H, W, 3) uint8 → (H, W) uint8  
    gray = 0.2126 * R + 0.7152 * G + 0.0722 * B
```

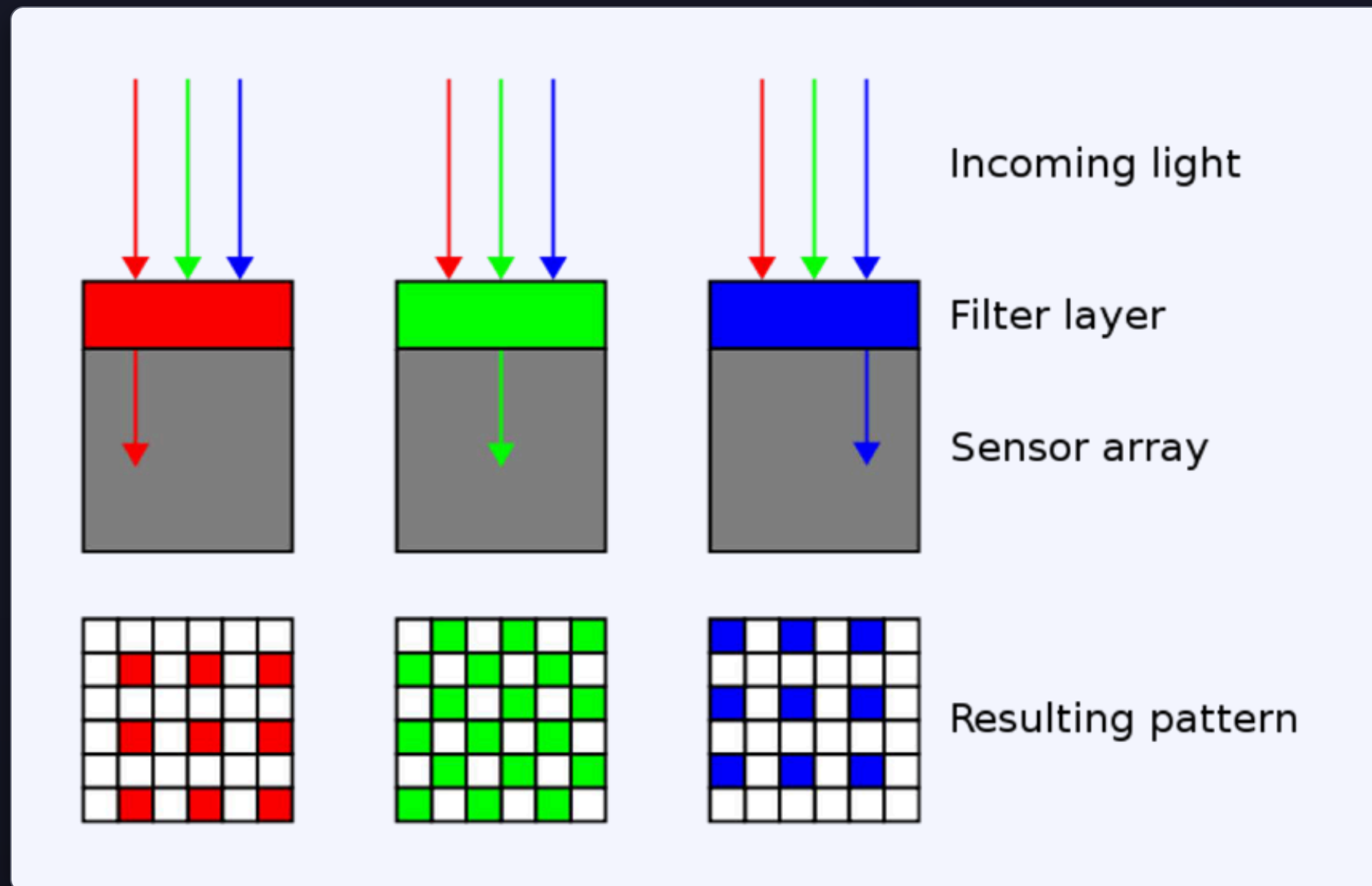


input



rgb_to_gray

A camera sensor records one colour per pixel



G	B	G	B
R	G	R	G
G	B	G	B
R	G	R	G

the GBRG pattern used in this assignment

- One colour filter per photosite, so the raw capture has a single channel. Half are green — the eye's most sensitive colour.
- Even rows: G B.
- Odd rows: R G.

simulate_bayer_mosaic and demosaic

CONCEPT

- A sensor measures one colour per pixel (GBRG, previous slide), giving a single-channel mosaic.
- Demosaicing interpolates each missing colour from neighbouring pixels.

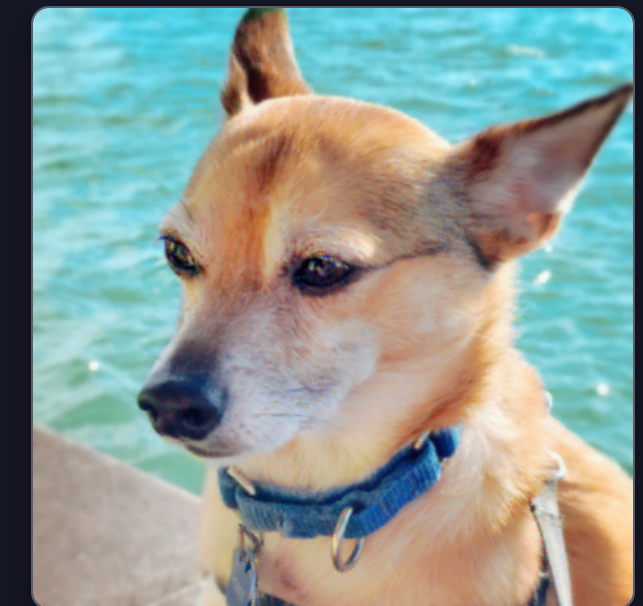
YOUR TASK — 5 + 5 MARKS

- `simulate_bayer_mosaic(rgb): (H, W, 3) → (H, W)`, keeping one channel per pixel following GBRG.
- `demosaic(bayer): (H, W) → (H, W, 3)`; keep the sampled colour, average neighbours for the other two.
- Work in float. Border pixels: "do something reasonable".

```
# strided slicing picks every other pixel:  
rgb[0::2, 1::2, 2] # blue: even rows, odd cols
```

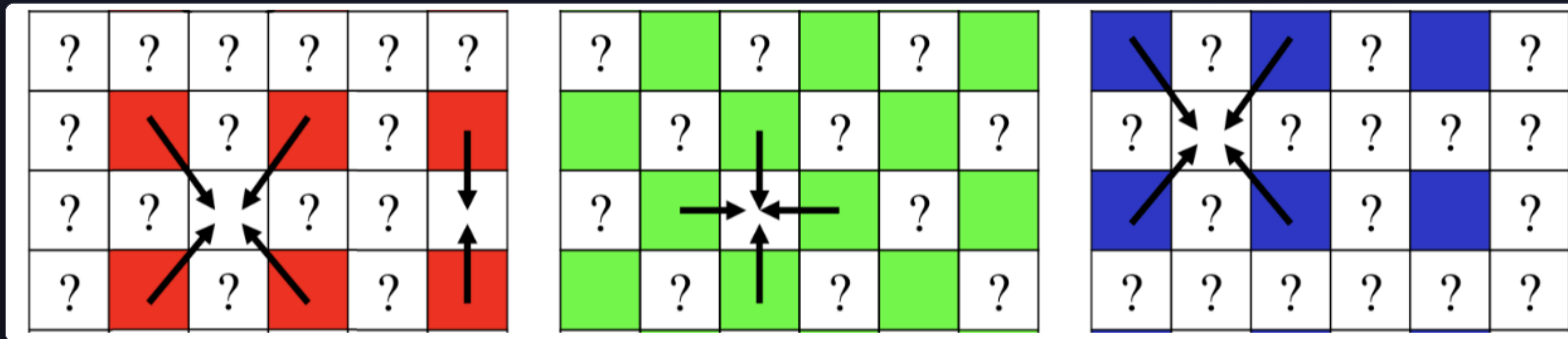


simulate_bayer_mosaic



demosaic

Demosaic: fill in each missing colour from its neighbours



- Missing red or blue: average the four diagonal neighbours (the "x"), or the two along the row or column. Missing green: average the four up/down/left/right neighbours (the "cross").
- This diagram's layout is not GBRG — in our pattern the top-left pixel is green.

rgb_to_hsv and hsv_to_rgb

CONCEPT

- HSV describes a colour by hue (which colour, in degrees 0–360), saturation (how vivid, 0–1) and value (how bright, 0–1).
- Filters like "shift the warmth" or "reduce saturation" are one-line edits in HSV but awkward in RGB.

YOUR TASK — 10 + 10 MARKS

- `rgb_to_hsv(r, g, b)` returns `(h, s, v)`; `hsv_to_rgb(h, s, v)` returns `(r, g, b)`. RGB values are in 0–1.
- Inputs may be scalars or whole numpy arrays — vectorize with `np.where` / `np.select`.
- Guard the divisions where chroma `C == 0` or `v == 0`.
- Do these first: `hue_shift` and `desaturate` import them.

HINTS FROM THE STUBS

```
def rgb_to_hsv(r, g, b):      # → (h, s, v)
    M = max(r, g, b);  m = min(r, g, b)
    C = M - m
    v = M
    s = C / v            # 0 when v == 0
    # h depends on which channel is M

def hsv_to_rgb(h, s, v):      # → (r, g, b)
    C = v * s;  hh = h / 60
    X = C * (1 - abs(hh % 2 - 1))
    m = v - C
    # six cases by interval of hh
```

hue_shift and desaturate

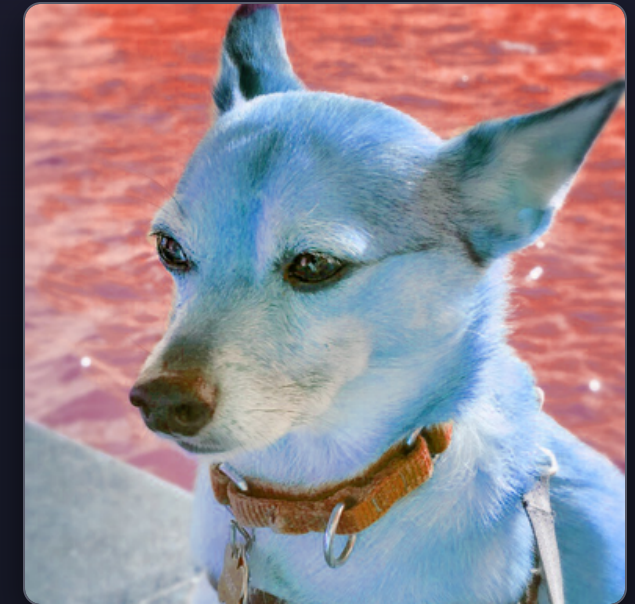
CONCEPT

- In HSV, rotating every hue recolours the image; scaling saturation pulls it toward gray.
- Hue is periodic: $350^\circ + 20^\circ = 10^\circ$.

YOUR TASK — 10 + 5 MARKS

- Both: float 0–1 $\rightarrow$ `rgb_to_hsv` $\rightarrow$ change one channel $\rightarrow$ `hsv_to_rgb` $\rightarrow$ back to 0–255.
- `hue_shift(rgb, shift)`: add shift degrees to every hue and wrap into 0–360.
- `desaturate(rgb, factor)`: multiply every saturation by $(1 - \text{factor})$; 0 leaves the image unchanged, 1 makes it gray.

```
# main.py calls
hue_shift(rgb, 180.0)
desaturate(rgb, 0.25)
```



hue_shift, 180°



desaturate, 0.25

over — alpha compositing

CONCEPT

- Compositing layers a partly transparent image A over B: opaque A shows A, transparent A shows B, in between a blend.
- Porter-Duff "over" is the exact formula, including the combined alpha.

YOUR TASK — 5 MARKS

- `over(A, B)`: composite RGBA A over RGBA B, both (H, W, 4) uint8, and return an (H, W, 4) uint8 result.
- Convert alphas to 0–1 first. Where the result alpha C_a is 0, set the colour to 0 instead of dividing.

$$C_a = A_a + B_a * (1 - A_a)$$

$$C_c = (A_a * A_c + B_a * (1 - A_a) * B_c) / C_a \quad \# \ 0 \text{ where } C_a == 0$$



`main.py` composites the three layers below over the photo



04

— Grading and debugging

How the mark is computed

Public tests — 30% of each function's marks

- These are the checks shipped in `tests/` — exactly what `run_tests.py` and `check_my_work.py` run.
- Their inputs and expected outputs are readable in `tests/expected/`.

Private tests — 70% of each function's marks

- The same checks, run on different input images.
 - They exist to catch code that hard-codes the published answers.
 - A correct implementation needs nothing beyond the public data to pass them.
-
- Do not copy code from the internet or from AI tools. Read explanations (Wikipedia is fine) and cite your references in comments.

Reading a failing image test

The whole image is slightly off

- A wrong coefficient or number range.
- Check the weights, 0–1 vs 0–255, and how you round back to uint8.

Only a few edge pixels differ

- A boundary case.
- Usually demosaic, where border pixels have fewer neighbours.

The image is mirrored or transposed

- An index-order mistake: rows vs columns.
- Or rotate going the wrong way.

```
out/test_diffs/<function>.got.png      # what your code produced
out/test_diffs/<function>.expected.png  # what was expected
out/test_diffs/<function>.diff.png      # where they differ (brighter = worse)
```

Questions?

A1 GitHub Issues · csc317tas@cs.toronto.edu